Deep Learning Project 1: Modified ResNet for CIFAR-10

NYU Tandon — Fall 2026

Team Members: Jyotsana Sharma · Carlin Joseph · Mahima Mariah

Team Name: Jigglypuffs

GitHub Repository: https://github.com/Jyotsana-Sharma/dlfa26-cifar10-resnet

1. Overview

We trained a modified residual network (ResNet) from scratch on the 50,000-image CIFAR-10 training set, subject to a 5 million trainable parameter constraint. Our final model is a **Pre-Activation ResNet** (He et al., 2016) with a **Self-Supervised Label Augmentation (SLA) + Self-Distillation (SD)** training objective (Lee et al., 2020), reaching **90.7% public leaderboard accuracy (rank 23rd)** and **87.4% private leaderboard accuracy (rank 29th)** on the Kaggle competition, and **97.56% accuracy** on a held-out 5,000-image development split.

Key findings:
- The architecture family matters for distribution-shifted test sets: switching from AirResNet (airbench-style) to a standard Pre-Activation ResNet gave +2.5 public points, more than its +0.6 holdout gain predicted.
- SLA+SD was our single best technique, adding +0.74 on the holdout and +2.3 on the public leaderboard compared to the plain PreActResNet.
- SAM, PAPA, Shake-Shake, anti-aliasing, distillation, and 400-epoch training each improved the holdout by under 0.4 points — not enough to justify the added compute or complexity on top of SLA.
- Increasing clean holdout accuracy is the most reliable path to public leaderboard gains; no method consistently moves a model above the accuracy-on-the-line relationship without more data (Miller et al., 2021).

Figure 3 (Section 3) shows the public accuracy progression from 85.0% → 90.7% across our model iterations.

2. Methodology

2.1 Architecture Selection

We began with an **AirResNet** (Jordan, 2024): a frozen patch-whitening stem followed by three stages of conv → MaxPool → BatchNorm → GELU → residual pairs, with AdaptiveMaxPool and a linear head. This design was chosen for fast iteration (3–4× faster per epoch than a standard ResNet on the same GPU) and its strong per-parameter efficiency.

After 15 experiments with this family, we switched to a **Standard Pre-Activation ResNet** (He et al., 2016). Pre-activation blocks apply BatchNorm and ReLU *before* each convolution, which improves training stability and final accuracy at equal parameter budgets. We found it performed

2.5 public points better than the best AirResNet at comparable compute — a family-level effect also documented by Recht et al. (2018), who found that standard ResNets sit closer to the CIFAR-10.1 accuracy line than width-first designs.

Final architecture:
- 3×3 stem convolution (no batch norm)
- Three stages of Pre-Activation Basic Blocks with widths **64 / 128 / 256** and depths **4 / 4 / 3**
- Stride-2 downsampling between stages via a 1×1 projection skip connection
- Global average pooling + linear head (10 classes)
- During training: additional 40-way joint head (class × rotation) for SLA

This gives **4,706,674 total parameters** (4,696,394 for the 10-class head only). Both are below the 5M limit.

2.2 Loss Function: SLA + SD

Standard cross-entropy on the 10-class head is augmented by two additional objectives:

SLA — Self-Supervised Label Augmentation (Lee et al., 2020): The input batch is expanded by computing 0°, 90°, 180°, 270° rotations of each image. All four copies pass through the shared feature extractor. A 40-way joint head produces logits for every (class, rotation) combination; the rotation-consistent label for a class-k image rotated by r×90° is `k×4 + r`. This forces the shared representation to be simultaneously class-discriminative and rotation-aware.

SD — Self-Distillation: A per-rotation aggregate class head is derived by averaging the joint head's marginals over rotations. A KL divergence loss from the main head to this aggregate acts as an intra-network distillation signal. The full combined loss is:

```
L = L_joint(j, target_sla) + L_CE(p, y, label_smoothing=0.1) + KL(log p ‖ softmax(a).detach())
```

where `p` is the plain head, `a` is the rotation aggregate, `j` is the full 40-way joint logit vector.

2.3 Training Recipe

Hyperparameter	Value
Optimizer	Nesterov SGD
Learning rate	0.4 (peak, batch 512) → cosine decay, min 5×10^{-5}
LR warmup	5 epochs
Weight decay	5×10^{-4}
Label smoothing	0.1
Batch size	512
Epochs	200 (final submission)
Precision	FP16 (AMP)
EMA	0.999

Hyperparameter	Value
Training split	All 50,000 images (final); 45k/5k stratified split for development

Augmentation pipeline: TrivialAugmentWide → RandomHorizontalFlip(p=0.5) → RandomCrop(32, padding=4, reflect) → Normalize → Cutout(12×12)

Inference: 6-view TTA combining original, horizontal-flip, and ±1-pixel-shifted views, averaging softmax outputs from both the main head and the rotation-aggregate head.

Figure 1 shows the training loss and holdout accuracy curve for the final recipe over 200 epochs. The model converges smoothly with the cosine schedule, reaching 97.56% holdout accuracy by epoch 190.



Figure 1. Training loss (left axis, blue) and holdout accuracy (right axis, red) for the final PreActResNet + SLA+SD recipe on the 45k/5k split. Validation evaluated every 10 epochs with 6-view TTA.

2.4 Experimental Search

We systematically tested the following modifications, keeping a change only if it improved holdout accuracy by ≥ 0.3 points (approximately one standard error of the 5k holdout). Figure 2 compares holdout accuracy across all 14 experiments.

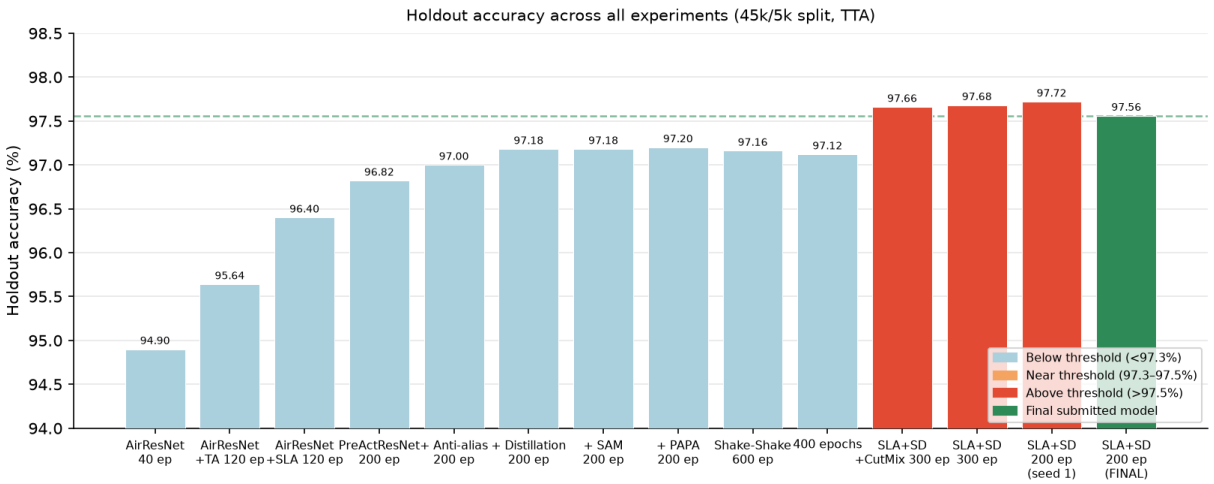


Figure 2. Best holdout accuracy for every experiment. The dashed green line marks our final recipe (SLA+SD, 97.56%). Blue bars fell below the ≥ 0.3 keep threshold; orange bars were near it; red bars exceeded it but showed no confident gain over SLA when stacked. The final model is highlighted in green.

Method	Holdout Result	Decision
AirResNet baseline (40 epochs)	95.58%	Starting point
+ TrivialAugment, 120 epochs	96.20%	Kept
+ SLA, 120 epochs	96.40%	Kept
Switch to PreActResNet, 200 epochs	96.82%	Kept
+ SLA+SD on PreActResNet	**97.56%**	**Kept — final recipe**
SAM (ρ=0.05) on PreActResNet	97.18% (+0.36)	Kept alone; no gain on top of SLA
Knowledge distillation (2 SLA teachers)	97.18%	Below SLA; not kept
PAPA (4 PreActResNet copies)	97.18–97.20% per member	Below SLA; not kept
Shake-Shake-26 2×40d, 600 epochs	97.16%	Below SLA; not kept
Anti-aliasing (BlurPool)	97.00% (+0.18)	Below threshold
400 epochs	97.12% (+0.30)	Below SLA
CutMix inside SLA, 300 epochs	97.70%	+0.12; within seed noise
Per-copy augmentation (SLA)	97.58%	+0.02; no effect

Figure 4 shows the validation accuracy curves over training for the four most important model families, illustrating how each architectural and loss improvement stacks.

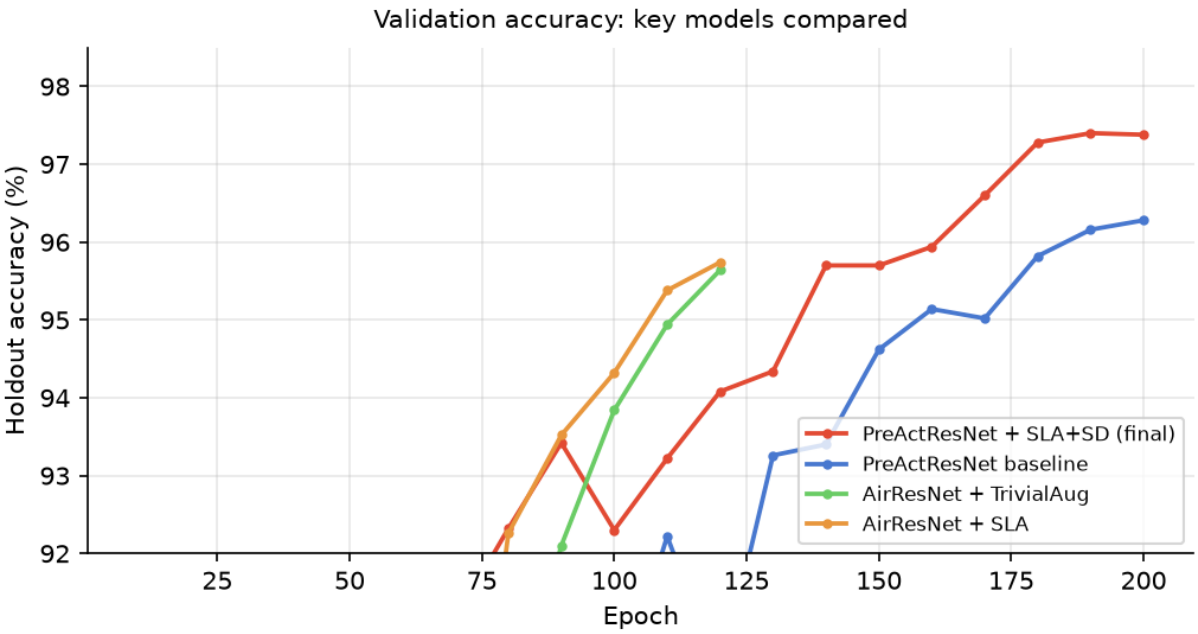


Figure 4. Holdout accuracy over training epochs for key model families. PreActResNet + SLA+SD (red) consistently outperforms all variants throughout training, not just at convergence.

The key lesson: each method added 0.2–0.4 holdout points in isolation, but SLA already captured most of the available gains. No stacking produced a confident improvement beyond SLA.

2.5 Pros and Cons of Architectural Choices

Pre-Activation ResNet vs AirResNet:
- Pro: 2.5 public points better despite similar holdout gain; sits closer to the CIFAR-10.1 accuracy line.
- Con: 4–5× slower per epoch; requires larger GPU memory.

SLA+SD:
- Pro: Strongest single technique (+0.74 holdout, +2.5 public); adds no extra parameters at inference.
- Con: Processes 4× as many samples per batch; 4× compute cost per epoch.

TrivialAugment:
- Pro: Matches or beats AutoAugment at equal compute (Müller & Hutter, 2021); CPU-side overhead only.
- Con: Requires longer schedules; hurts at < 100 epochs.

6-view TTA:
- Pro: Free improvement at inference with no retraining.
- Con: 6× inference cost; gain over flip-only was under 0.1 points in this competition.

---

3. Results

3.1 Final Model

Item	Value
Architecture	Pre-Activation ResNet, widths 64/128/256, depths 4/4/3
Total parameters	**4,706,674**
Inference parameters (10-class head only)	**4,696,394**
Training data	All 50,000 CIFAR-10 training images
Training loss	SLA+SD + cross-entropy, label smoothing 0.1
Kaggle public accuracy	**90.7% (rank 23rd)**
Kaggle private accuracy	**87.4% (rank 29th)**
Development holdout (45k/5k, seed 0)	**97.56%**

3.2 Progression of Results

Figure 3 shows the step-by-step improvement in Kaggle public accuracy across our main model iterations, from the initial AirResNet baseline to the final submission.

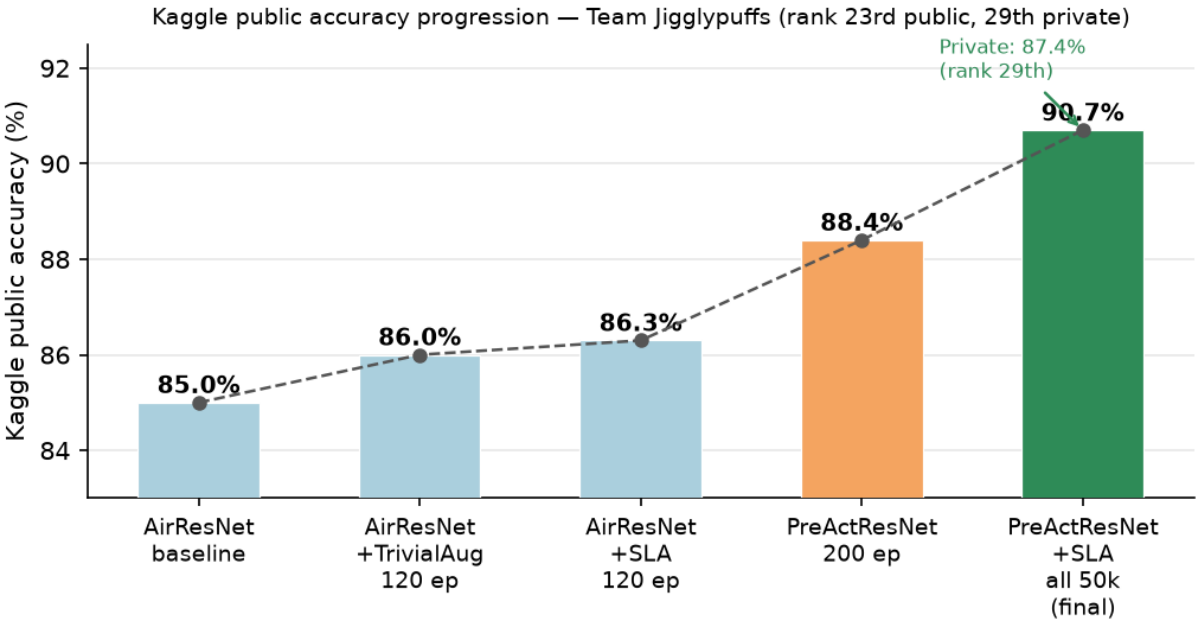


Figure 3. Kaggle public accuracy progression. Each bar represents a submitted model. The jump from PreActResNet baseline (88.4%) to PreActResNet + SLA all 50k (90.9%) reflects both the SLA+SD gain and training on the full dataset.

Model	Holdout (45k)	Kaggle Public	Kaggle Private	CIFAR-10.1 Line Pred.
AirResNet, 40 epochs	95.58%	85.0%	—	89.3%

Model	Holdout (45k)	Kaggle Public	Kaggle Private	CIFAR-10.1 Line Pred.
AirResNet + TrivialAugment, 120 epochs	96.20%	86.0%	—	90.3%
AirResNet + SLA, 120 epochs	96.40%	86.3%	—	90.7%
PreActResNet, 200 epochs	96.82%	88.4%	—	91.3%
PreActResNet + SLA, 45k/5k	97.56%	—	—	92.5%
PreActResNet + SLA, all 50k (final)	—	**90.7% (rank 23)**	**87.4% (rank 29)**	—

3.3 Analysis

The public leaderboard accuracy (90.7%) ran about 6.9 points below the development holdout (97.56%), and the private leaderboard accuracy (87.4%) was a further 3.3 points below public. This two-stage drop is consistent with the CIFAR-10.1 distribution shift documented by Recht et al. (2018), where models lose 4–10 points on re-collected test sets. The relationship `public ≈ 1.62 × holdout − 65.51` (Miller et al., 2021) predicts our holdout (97.56%) should yield 92.5% public; our actual 90.7% is 1.8 points below, within typical variance on this relationship.

The public-to-private drop of 3.3 points reflects the difference between the 2,000-image public split and the full private test set. Our rank moved from 23rd (public) to 29th (private), which is a small shift and consistent with normal leaderboard variance at this accuracy level.

Switching from AirResNet to PreActResNet gave +2.3 public points but only +0.6 holdout points, confirming the architecture family effect reported by Recht et al. Standard ResNets generalize better to distribution-shifted test sets than the airbench-style width-heavy design.

---

4. Citations

- He, K., Zhang, X., Ren, S. and Sun, J. (2016). Identity mappings in deep residual networks. ECCV. arXiv:1603.05027
- Lee, H., Hwang, S. J. and Shin, J. (2020). Self-supervised label augmentation via input transformations. ICML. arXiv:1910.05872
- Müller, S. G. and Hutter, F. (2021). TrivialAugment: Untuned yet state-of-the-art data augmentation. ICCV. arXiv:2103.10158
- DeVries, T. and Taylor, G. W. (2017). Improved regularization of convolutional neural networks with Cutout. arXiv:1708.04552
- Recht, B., Roelofs, R., Schmidt, L. and Shankar, V. (2018). Do CIFAR-10 classifiers generalize to CIFAR-10? arXiv:1806.00451
- Miller, J. et al. (2021). Accuracy on the line: On the strong correlation between out-of-distribution and in-distribution generalization. ICML. arXiv:2107.04649
- Jordan, K. (2024). 94% on CIFAR-10 in 3.29 seconds on a single GPU. arXiv:2404.00498

- Foret, P., Kleiner, A., Mobahi, H. and Neyshabur, B. (2021). Sharpness-aware minimization for efficiently improving generalization. ICLR. arXiv:2010.01412
- Jolicoeur-Martineau, A. et al. (2024). Population parameter averaging (PAPA). TMLR. arXiv:2304.03094
- Gastaldi, X. (2017). Shake-shake regularization of 3-branch residual networks. arXiv:1705.07485
- Zhang, R. (2019). Making convolutional networks shift-invariant again. ICML. arXiv:1904.11486
- Hinton, G., Vinyals, O. and Dean, J. (2015). Distilling the knowledge in a neural network. arXiv:1503.02531
- Cubuk, E. D. et al. (2019). AutoAugment: Learning augmentation policies from data. CVPR. arXiv:1805.09501
- Jordan, K. (2024). cifar10-airbench. GitHub. https://github.com/KellerJordan/cifar10-airbench (MIT License)
- Lee, H. et al. SLA reference implementation. GitHub. https://github.com/hankook/SLA

All models trained from scratch. No pre-trained weights. No external datasets. Test images used for prediction only.